CYBERSTR RED TEAM INTERNSHIP • STUDY NOTES

Denial of Service Attacks Explained & Demonstrated

DoS, DDoS, reflection, amplification, volumetric, protocol, and application layer attacks — with real tool commands

Prepared for: Goraya — Red Team Intern, CyberStr

Trainer: Umar Niaz

Source: ITProTV – PenTest+ Series (Daniel Lowrie & Ronnie)

Level: Junior Penetration Tester

Topics: DoS vs DDoS, Reflected/Amplified attacks, hping3, Smurf & Fraggle, Ping of Death, UDP Flood, Slowloris (Metasploit)

Notes compiled and expanded for practical, exam-and-lab-ready learning. Explanatory + reasoning style, plain English.

00 Table of Contents

01 Introduction

*This lesson covers Denial of Service (DoS) attacks — a heavily tested PenTest+ exam topic, and also a real-world engagement type some clients specifically request under the name **stress testing** or **stress assessment**.*

WHY THIS MATTERS

Not every client wants a DoS test — it can be disruptive. But some clients specifically ask, "can our infrastructure survive being hammered?" As a pen tester, you need to understand the different categories of DoS attacks, the tools used to perform them in a lab, and how to explain the impact to a client in a report.

The lesson walks through these categories, from general to specific:

DoS vs DDoS | Reflected | Amplified | Volumetric | Protocol | Application Layer

02 DoS vs DDoS — The Basic Difference

2.1 Denial of Service (DoS)

A **Denial of Service** attack is exactly what it sounds like: denying a type of service to legitimate users. It is straightforward and self-explanatory.

EXAMPLES FROM THE LESSON

- A web service that nobody can reach → denial of service.
- An FTP site where no one can upload or download files → denial of service.

2.2 Distributed Denial of Service (DDoS)

A **Distributed** Denial of Service attack is different in one key way: instead of a single attacker doing their best to cripple the infrastructure alone, many devices ("friends," in the lesson's analogy) join together and attack the same target at once.

REASONING — WHY "DISTRIBUTED" MATTERS

The best one-line definition given in the lesson: "*there's more strength in numbers.*" Multiple devices attacking a single target simultaneously are far more effective than a single device attacking alone — both because more traffic can be generated, and because blocking one attacking source doesn't stop the rest.

IMPORTANT CLARIFICATION

DoS and DDoS are **descriptions**, not a single specific technology or exact technique. Literally anything that results in denying legitimate service counts. This is why there are so many different sub-types — reflected, amplified, volumetric, protocol-based, application layer — each is simply a different *method* of achieving that same general goal.

03 Reflected Attacks

A reflected attack is a technique for hiding the true source of an attack by "bouncing" it off of an innocent third party.

WALKTHROUGH EXAMPLE (DNS REFLECTION)

Instead of attacking the target directly — which creates an obvious, traceable straight-line path from attacker to victim — the attacker instead sends a request to a third party (in this example, a DNS server) and **spoofs the source address** to make it look like the request came from the victim, not the attacker.

The DNS server, believing the request genuinely came from the victim, sends its response **directly to the victim**. The victim now receives a flood of unsolicited DNS responses to queries it never actually made.

REASONING — WHY ATTACKERS USE REFLECTION

- **Hides the attacker's identity** — the traffic hitting the victim appears to originate from the reflecting server (e.g. the DNS server), not the real attacker.
- **Uses a legitimate service against itself** — the DNS server is simply doing its normal job (answering queries); it is not compromised, it's being tricked into becoming a weapon.
- Combine this with many reflecting servers ("Ronnie and all his DNS buddies") and the effect multiplies — this naturally leads into amplification (next section).

PEN TESTER TAKEAWAY

Reflection relies on **IP address spoofing** — forging the source address of a packet. Recognizing reflected DDoS in the wild (DNS reflection, NTP reflection, etc.) means looking for large volumes of unsolicited response traffic from legitimate-looking third-party servers.

04 Amplification Attacks

Amplification is often combined with reflection (as in the DNS example above) to make the attack far more damaging per request sent.

KEY TERMS FROM THE LESSON

- **Trigger packet** — the small initial query sent to the reflecting server (e.g. the DNS query).
- **Payload** — the much larger response that the reflecting server sends back to the target.

With DNS, the response the server generates is typically **much larger** than the original query — giving the attacker "more bang for the buck." Instead of crafting a large malicious packet from scratch, the attacker gets the DNS server to generate a large packet on their behalf and send it to the target.

REASONING — WHY AMPLIFICATION IS EFFICIENT FOR AN ATTACKER

Sending one small spoofed query can trigger a response many times larger being delivered to the victim. This means the attacker needs far less of their own bandwidth to generate a devastating amount of traffic hitting the target — an "amplification factor" or multiplier effect. Combined with many reflecting servers responding at once, the resulting flood can be enormous compared to what the attacker originally sent.

05 Volumetric Attacks

*A **volumetric attack** is about raw quantity: sending so much data that it saturates the target's network bandwidth and overwhelms its ability to service requests.*

> **CORE IDEA**
>
> The classic example given is a **volumetric ping** — sending enough ping traffic that it saturates the network pipe and the target's ability to withstand requests. It's not about tricking the protocol into breaking (that's a protocol attack, covered later); it's simply about overwhelming with sheer volume.

5.1 What is hping3?

hping3 is described as "ping on steroids" — a far more capable tool than the standard `ping` command, offering raw packet manipulation and many advanced options for crafting exactly the kind of traffic you want to send.

```
hping3 --help
```

5.2 Key hping3 Options (from the --help output)

Option	What It Does (as explained in the lesson)
`--count`	Sets how many packets to send — useful for controlling the size/duration of a DoS attempt.
`--interval`	Controls how long the tool waits before sending the next packet.
`--flood`	Sends packets as fast as possible and does not wait to show replies — ideal for maximum-speed flooding.
`--interface`	Lets you choose which network interface to send from, if you have multiple.
Raw IP / `--icmp` / `--udp`	Different packet modes — choose the underlying protocol used to build the packets.
`--scan`	A scan mode, with a built-in usage example shown in the help output.
`-a` / `--spoof`	Spoofs the source address, so the traffic does not appear to come from the real attacker.

Random source / destination	Lets you use a randomized address instead of a specific one, to further obfuscate origin.
Fragmentation options	Useful for trying to get packets past firewalls that inspect traffic.
`--mtu`	Adjusts the Maximum Transmission Unit — useful depending on the type of service/attack.
ICMP type options	Different ICMP types can be selected; useful because some specific types may be blocked by defensive mechanisms while others pass through.
Flag-setting options	Full packet-crafting control — you can set custom TCP flags and build very specific crafted packets.

5.3 Live Demo Walkthrough

LAB SETUP

The target in the demo is a web server running an old, vulnerable web application (the lesson mentions it running on a "Metasploitable 2"-style vulnerable box), accessed and monitored through a browser while the attack runs.

STEP-BY-STEP COMMAND BREAKDOWN

```
sudo hping3 --flood --spoof 192.168.241.133 --icmp 192.168.241.130
```

- `sudo` — required because packet crafting needs low-level control (raw sockets).
- `--flood` — sends packets as fast as possible without waiting for replies.
- `--spoof 192.168.241.133` — forges the source IP so it does not look like it came from the attacker's real address.
- `--icmp` — sends the flood using the ICMP protocol (like an aggressive ping).
- `192.168.241.130` — the intended target IP address.

RUNNING IT IN THE BACKGROUND

After launching the command, the instructor stops it in the foreground with `Ctrl+C`, then re-launches it with an ampersand at the end so it runs in the background, freeing up the terminal for other work (like monitoring):

```
sudo hping3 --flood --spoof 192.168.241.133 --icmp 192.168.241.130 &
```

This allows opening a second terminal to run a normal ping and observe the target's response times live while the flood runs in the background.

MONITORING THE EFFECT

```
ping 192.168.241.130
```

Observed in the demo:

- At first, response times stayed under 1 millisecond — the target still felt "snappy."
- Running the flood command multiple times (stacking multiple flood processes against the same target) gradually increased response times — up to 4ms, then 5-6ms, then higher.
- After stacking several more flood processes (the instructor ran it repeatedly, several times over), response times climbed dramatically — eventually into the 37-47ms range and the website became visibly slow and unresponsive in the browser, ultimately resulting in timeout errors.

REASONING — WHY RESPONSE TIME IS THE SIGNAL

As more flood processes pile up, the target's CPU, network stack, and bandwidth become increasingly consumed just trying to process the flood of ICMP packets. This leaves fewer resources to service legitimate requests (like the ping test, or a real browser request), which is exactly why response times climb and the website becomes sluggish or unreachable — the practical, observable effect of a volumetric DoS attack.

CLEANING UP — STOPPING ALL BACKGROUND JOBS

With several hping3 flood jobs running as background jobs, the instructor lists and kills them all using a simple shell loop:

```
jobs
# lists all running background jobs, e.g. job numbers 1 through 7

for i in %1 %2 %3 %4 %5 %6 %7; do kill $i; done
```

- `jobs` — lists currently running background jobs in the current shell session.
- `%N` — refers to background job number N.
- The `for` loop iterates through each job reference and issues `kill` against it, stopping the flood processes one by one.

After killing the jobs, the target's response time returns to normal, confirming that the hping3 floods were the direct cause of the slowdown.

5.4 Smurf & Fraggle Attacks

These are two closely related, classic volumetric/amplification-style attacks that both abuse a network's **broadcast address**.

SMURF ATTACK

Conceptually similar to an ICMP flood, but instead of targeting a specific host directly, the attacker sends the traffic to the network's **broadcast address**. Every single device on that network segment is obligated to reply to a broadcast address contact — meaning one spoofed request can trigger replies from many devices at once, all flooding toward the spoofed (victim's) source address.

FRAGGLE ATTACK

Works on the same underlying idea as Smurf, but uses **UDP** instead of ICMP, and specifically targets:

- **Port 7** — the Echo port
- **Port 19** — the Chargen (character generator) port

instead of relying on ICMP echo requests.

REASONING — WHY THE BROADCAST ADDRESS IS THE KEY

A broadcast address is designed so that **every host** on the local network segment receives and responds to the message. This is exactly what makes it dangerous when abused — a single spoofed packet sent to a broadcast address can generate replies from dozens or hundreds of devices simultaneously, all directed at the spoofed victim address. This is effectively a "free" amplification multiplier baked into how broadcast addressing works.

06 Protocol Attacks

*A **protocol-based** denial of service attack works differently from a pure volumetric flood: instead of just overwhelming with quantity, it exploits how a specific protocol is designed or implemented to actually break or crash the target.*

6.1 Ping of Death

HOW IT WORKS

The classic "Ping of Death" involves crafting an **oversized packet** — larger than what the receiving device is able to properly process. When the target receives this malformed, oversized packet and attempts to process it, the device **crashes** (a classic example given is causing a Windows machine to blue-screen).

REASONING — WHY CRASHING THE MACHINE COUNTS AS DOS

The lesson makes the logic explicit: a crashed machine does not respond to further requests at all. If the attacker can crash the target outright, that is a complete (if often temporary, until reboot) denial of service — no ongoing flood of traffic is even needed once the crash occurs. This is why it's called "protocol-based" — the attacker used a misconfiguration/flaw in how the protocol handles oversized packets to force the system into submission and crash it.

6.2 UDP Flood

HOW IT WORKS

A UDP flood manipulates how the UDP protocol itself behaves. Because UDP handles open vs. closed ports differently than TCP, each incoming UDP packet aimed at a closed port has to be **serviced** (processed and responded to, typically with an ICMP "port unreachable" message) by the target machine.

REASONING

By flooding a large number of UDP requests at closed ports, the attacker forces the target to spend significant resources just processing and responding to all of these requests. The machine becomes overwhelmed trying to service this artificial load — again exploiting how the protocol is designed to behave, which is why this also falls under "protocol-based" denial of service.

07 Application Layer Attacks

Application layer attacks target the protocols and mechanics of the application layer itself — commonly HTTP/HTTPS for web services — rather than lower-level network protocols.

FOUNDATION — HOW NORMAL WEB REQUESTS WORK

Every time a browser loads a page, it's typically using an HTTP **GET** request (asking the server to return a page) or a **POST** request (submitting data to be processed). These are the standard, everyday methods that make the web work — and it's precisely this everyday mechanism that gets abused in an application layer attack.

REASONING — WHAT HAPPENS UNDER OVERLOAD

If a target is bombarded with a very large number of HTTP requests, its ability to keep up depends on available bandwidth, RAM, and CPU. Once these resources are exhausted trying to service the flood of requests, the application layer itself effectively breaks down — a denial of service achieved without necessarily needing a network-layer flood or a protocol-level crash.

7.1 Accidental Real-World DoS (Reddit Hug of Death)

The lesson points out that application-layer DoS conditions can happen completely by accident, with no attacker at all.

REAL EXAMPLES MENTIONED

- When Michael Jackson died, the surge of simultaneous searches reportedly overwhelmed services like Twitter and Google.
- The **"Reddit hug of death"** — when a link is heavily discussed and shared on Reddit, the resulting flood of simultaneous clicks can overwhelm the linked site's server, effectively denying service to everyone trying to access it, purely from organic popularity.
- An older, similar term for the same phenomenon on a different platform was called the **"Digg effect"**.

PEN TESTER TAKEAWAY

This shows why application-layer resilience (proper load balancing, caching, auto-scaling, rate limiting) matters even outside of malicious attack scenarios — legitimate

traffic spikes can produce the exact same denial-of-service symptoms as a deliberate attack.

7.2 Slowloris (Metasploit Demo)

Slowloris is described in the lesson as a particularly effective and "evil" (in a fun, admiring sense) application-layer DoS technique, demonstrated using the Metasploit Framework.

STEP-BY-STEP COMMAND BREAKDOWN

```
msfconsole
# launches the Metasploit Framework console

search slowloris
# searches Metasploit's module database for the slowloris auxiliary module

use 0
# selects the first (top) search result returned

info
# displays details about the module

set RHOST 192.168.241.130
# sets the target host to attack

exploit
# (or "run") launches the module against the target
```

WHAT THE MODULE'S INFO DESCRIBED (AS READ IN THE LESSON)

The Slowloris module **tries to keep many connections to the target web server open** and hold them open for as long as possible. It accomplishes this by opening connections and sending **partial HTTP requests** periodically — continuing to add additional HTTP headers over time, but **never actually completing the request**. Affected servers keep these incomplete connections open, which eventually fills up the server's maximum concurrent connection pool, denying any further connection attempts from real clients.

REASONING — WHY THIS IS SO EFFECTIVE WITH SO LITTLE TRAFFIC

Unlike a volumetric flood (which needs huge amounts of bandwidth), Slowloris works by exploiting the server's connection-handling logic itself — specifically, that it patiently waits for a request to finish before freeing up that connection slot. By never finishing the request, the attacker can occupy connection slots almost indefinitely

using only a small trickle of traffic. The lesson notes this makes it "a whole lot easier" to pull off than a raw hping3-style volumetric flood, while being extremely effective at denying new connections.

PEN TESTER TAKEAWAY — THE THREE-WAY HANDSHAKE CONNECTION

This attack takes advantage of the same idea as a TCP three-way handshake being left incomplete: the server allocates resources for a connection expecting it to finish, and if it never does, that resource sits tied up. Recognizing "connections opened but never finishing" as a Slowloris-style symptom is an important diagnostic skill.

08 Quick Reference & Cheat Sheet

DoS Attack Categories — At a Glance

Category	Core Idea	Example From Lesson
DoS	Single attacker denies service to legitimate users	Web service/FTP site unreachable
DDoS	Many devices attack the same target together	Multiple devices flooding one target
Reflected	Spoof victim's IP, trick a third party into responding to the victim	DNS query spoofed to look like it came from the victim
Amplified	Small trigger packet causes a much larger payload to be sent	DNS response much larger than the DNS query
Volumetric	Overwhelm with sheer traffic volume / bandwidth saturation	hping3 ICMP flood; Smurf/Fraggle broadcast abuse
Protocol	Exploit how a protocol is designed/ implemented to break the target	Ping of Death (oversized packet); UDP flood
Application	Exhaust app-layer resources (bandwidth/RAM/CPU) or hold connections open	HTTP request floods; Slowloris

Command Reference

```
# View all hping3 options
hping3 --help

# Volumetric ICMP flood with spoofed source, run in background
sudo hping3 --flood --spoof <fake-source-ip> --icmp <target-ip> &

# Monitor target responsiveness from a second terminal
ping <target-ip>

# List and kill all running background flood jobs
jobs
for i in %1 %2 %3 %4 %5 %6 %7; do kill $i; done

# Slowloris application-layer attack via Metasploit
msfconsole
```

```
search slowloris
use 0
info
set RHOST <target-ip>
exploit
```

Checklist — Running a DoS Test Safely in a Lab

- Confirm you are authorized to test the target (DoS testing is high-impact — always get explicit client sign-off/scope).
- Use an isolated lab target (e.g. Metasploitable 2) — never test unauthorized live systems.
- Monitor the target's responsiveness (e.g. via `ping`) so you can measure and report actual impact.
- Track every background process you start (`jobs`) so you can cleanly stop the test afterward.
- Understand the mechanism behind each attack type (reflection, amplification, protocol flaw, connection exhaustion) so you can explain root cause and remediation in your report, not just "it went down."

09 Key Takeaways for a Junior Pen Tester

1. DOS/DDOS ARE DESCRIPTIONS, NOT ONE TECHNIQUE

Any action that denies legitimate service counts. The many named sub-types (reflected, amplified, volumetric, protocol, application) are just different *methods* of achieving the same general goal — recognize the underlying method, not just the label.

2. SPOOFING IS THE COMMON THREAD IN REFLECTION/AMPLIFICATION

Reflected and amplified attacks both rely on forging (spoofing) the source IP address so the reflecting server's response goes to the victim instead of the real attacker. Recognizing spoofed-source traffic patterns is a key detection skill.

3. NOT EVERY DOS NEEDS MASSIVE BANDWIDTH

Slowloris proves that a clever attack exploiting how a protocol/server handles connections can be devastating using only a trickle of traffic — far easier to pull off than a brute-force volumetric flood like hping3.

4. CRASHING ≠ FLOODING, BUT BOTH DENY SERVICE

Ping of Death shows that denial of service doesn't always require sustained traffic — sometimes a single malformed packet that crashes the target achieves the same end result, instantly and completely (until the system is manually restarted).

> *"There's more strength in numbers... you are more effective when you have multiple devices lobbying the attack at your intended target than you do with just a single [device]." — Daniel Lowrie, on why DDoS is more powerful than DoS*

NEXT STEPS FOR YOUR OWN PRACTICE

- In your CyberStr lab, try the hping3 flood command against Metasploitable 2, and log the ping response-time change — this builds a real before/after dataset for a report.
- Run the Slowloris Metasploit module against a lab web server and observe connection exhaustion directly, comparing effort vs. impact against the hping3 approach.

- Practice explaining, in plain English, why each attack type works — this is exactly the kind of root-cause explanation clients expect in a professional DoS assessment report.